

zai-org/GLM-5.3-BF16 like 17 Follow Z.ai 19.6k

Text Generation
 Transformers
 Safetensors
 English
 Chinese
 glm_moe_dsa
 conversational
 arxiv:2602.15763
 License: glm-5.3

Deploy → Copy to bucket **NEW**
 Use this model

Model card
 Files
 xet
 Community 1

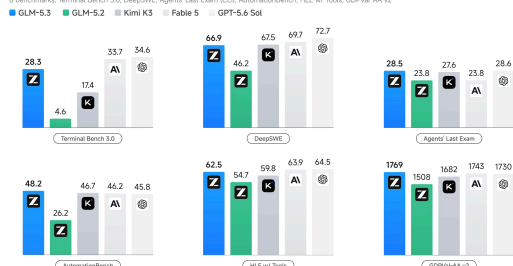
GLM-5.3-BF16

GLM-5.3 uses the same base model as GLM-5.2 — every gain comes from post-training. Compared with GLM-5.2, it is much better at complex coding and long-horizon tasks:

- **Stronger Coding:** GLM-5.3 is the most capable open-weights model for coding, with a 50% improvement over GLM-5.2 on our in-house Z.ai Code Bench. It also achieve open-source SOTA on public benchmarks including Terminal Bench 3.0 and Agents' Last Exam.
- **Emergent Cyber Capability:** As we scaled post-training, cyber capability developed faster than we expected. GLM-5.3 is state of the art on CyberGym for vulnerability discovery, and its gains are largest further up the exploitation chain, where it more than doubles GLM-5.2 on exploitation benchmarks.

LLM Performance Evaluation

6 benchmarks: Terminal Bench 3.0, DeepSWE, Agents' Last Exam (CJUL), AutomationBench, HLE w/ Tools, GDFair-AA v2



Benchmark

Downloads last month

Safetensors

Model size 753B params

Tensor type BF16 · F32

Chat template
 Files info

Inference Providers

Text Generation

Input a message with zai-org/

Run 15,000+ Models Instantly

Inference Providers let you run inference on thousands of models served by our partners using a simple, unified, OpenAI-compatible serverless API ([Learn more](#)).

zai-org/GLM-5.3-BF16 is supported by the following Inference Providers:

Zai

[View API Code](#)

[Dismiss](#)

Your prompt here...

Send

View Code

Compare providers

Model tree for zai-org/GLM-5.3-B...

Quantizations 3 models

Collection including zai-org/GLM-...

GLM-5.3 Collection

4 items • Updated 1 day ago • 24

Paper for zai-org/GLM-5.3-BF16

GLM-5: from Vibe Coding to Agentic...

Paper • 2602.15763 • Pub... • 216

Benchmark	GLM-5.3	GLM-5.2	Kimi K3	DeepSeek-V4 Pro-0813
Terminal Bench 2.1	88.2	81.0	88.3	87.9
Terminal Bench 3.0	28.3	4.6	17.4	–
DeepSWE (v1.1)	66.9	46.2	67.5	62.7
NL2Repo	58.0	48.9	58.0	61.1
ProgramBench (Almost Solved)	19.0	9.5	17.5	–
FrontierSWE	78.1	67.5	–	–
SWE-Marathon (v1.1)	42.5	19.4	48.1	–
PostTrainBench	39.8	31.7	32.0	–
CyberGym	84.5	77.2	80.0	83.3
ExploitGym (2h / 6h)	105 / 130	29 / 39	36 / 70	–
ExploitBench	54.4	24.4	32.2	–
Toolathlon Verified	73.0	59.9	76.5	74.1
AutomationBench (v1.0.6)	48.2	26.2	46.7	43.2
Agents' Last Exam (ALE-CLI)	28.5	23.8	27.6	25.7
HLE w/ Tools	62.5	54.7	59.8	60.0
GDPval-AA v2	1769	1508	1682	1590

Serve GLM-5.3 Locally

GLM-5.3 supports deployment with the following frameworks. Feel free to try them out:

- [SGLang](#) — see [cookbook](#)
- [vLLM](#) — see [recipes](#)
- [TokenSpeed](#) — see [here](#)
- [Transformers](#) — see [transformers docs](#)
- [KTransformers](#) — see [tutorial](#)
- [Unsloth](#) — see [guide](#)

- For deployment on the Ascend NPU platform, inference frameworks such as vLLM-Ascend, xLLM and SGLang are supported — see [here](#).

Note

- GLM-5.3 supports controlling the thinking budget through the `reasoning_effort` parameter, which accepts three levels: `low`, `high`, and `max`. It defaults to `max` if not passed (or if set to any other value). To use `low` or `high`, pass them explicitly. For benchmark and leaderboard reproduction, keep the default `max`.
- In the chat template for GLM-5.3, `clear_thinking` defaults to `false` if not passed. For chat scenarios, explicitly pass `clear_thinking=true`.

Footnotes

- **HLE w/ tools:** We use sampling parameters of `temperature=1.0` and `top_p=0.95` for evaluation, with a maximum generation length of 163,840 tokens. The evaluation is conducted with a maximum context length of 300,000 tokens, using a context management strategy. We use GPT-5.6-luna (medium) as the judge model.
- **NL2Repo:** We evaluated NL2Repo with `temperature=1.0`, `top_p=1.0`, and `max_new_tokens=64k` under 1M context. To prevent hacking, we use rule-based and a LLM-based judgement to prevent malicious behaviors (e.g., unauthorized pip or curl operations).
- **DeepSWE:** We run DeepSWE using the mini-swe-agent harness with `temperature=0.95`, `top_p=1.0`, `timeout=6h` and 400K context.
- **Terminal-Bench 2.1:** We evaluate in Claude Code 2.1.207 with `temperature=1.0`, `top_p=1`, `max_new_tokens=65536` with 6h timeout.
- **Terminal-Bench 3.0:** We evaluate Terminal-Bench-3 tasks with the Claude Code 2.1.207 harness (`reasoning_effort=max`, 400K context, and 128K maximum output), reporting avg@3 over three rollouts per task. Each rollout runs

in an isolated container built from the task's official image, and is capped at 600 agent turns with a 10-hour timeout. Tool Search is disabled, and the artifacts each agent produces are scored by the task's official separate verifier.

- **Agent's Last Exam (CLI):** We evaluate ALE using the official evaluation protocol with the Claude Code harness (reasoning effort=max, 1M context, and 64K maximum output). Each of the 105 tasks runs in an isolated Docker container using the resources declared in its Task Card. The default timeout is 4 hours, with task-specific limits taking precedence (up to 8 hours). Tool Search is disabled, and results are scored by the official ALE evaluators.
- **Toolathlon Verified:** We obtain all results via the official evaluation service and report pass@1 averaged over 3 independent runs.
- **AutomationBench:** We evaluate on AutomationBench **v1.0.6**, incorporating the fix for the null-type handling issue introduced in [PR #13](#).
- **GDPval-AA v2:** Models are evaluated by Artificial Analysis.
- **CyberGym:** We evaluate GLM-5.3 in Claude Code 2.1.207 (max reasoning effort, no web tools with temperature=1.0, top_p=1.0, max_new_tokens=128000). All evaluations are under unlimited timeout per task and results are single-run Pass@1 over 1,507 tasks. To simulate real-world usage scenarios, we place the agent inside the task container. We also remove all Git-related information and apply a domain whitelist (allowing only essential domains such as pypi.org and deb.debian.org for basic tool installation) to prevent the agent from cheating.
- **ExploitGym:** We evaluate GLM-5.3, Kimi-K3 and Qwen3.8 Max in Claude Code 2.1.207 (max reasoning effort, no web tools with temperature=1.0, top_p=1.0, max_new_tokens=128000). The reported results are single-run Pass@1 on 869 tasks under two timeout budgets: 2 hours and 6 hours, which are calculated as the API inference time rescaled by per-model tokens per second rate

(per-model TPS sourced from Artificial Analysis; that is, we rescale GLM-5.3's results by 115 TPS, Kimi K3's results by 40 TPS and Qwen3.8 Max's results by 47 TPS), plus the non-API overhead. We also apply a domain whitelist (allowing only essential domains such as pypi.org and deb.debian.org for basic tool installation) to prevent the agent from cheating.

- **ExploitBench:** We evaluate GLM-5.3 in Claude Code 2.1.207 (max reasoning effort, no web tools with `temperature=1.0`, `top_p=1.0`, `max_new_tokens=128000`). Following the official evaluation settings, we limit the maximum number of interaction rounds between the agent and the environment to 300, and compute the average coverage score over all 41 tasks across 3 revisions. The coverage result of a task is determined by taking the union of capabilities achieved across all revisions, and the average score is obtained by averaging the results. We also apply a domain whitelist (allowing only essential domains such as pypi.org and deb.debian.org for basic tool installation) to prevent the agent from cheating.
- **FrontierSWE:** The evaluation was conducted by [Proximal](#) with 1M context length, max effort level, and 128K maximum output tokens. Dominance score reported as of 2026/08/14.
- **PostTrainBench:** We evaluate GLM-5.3 using Claude Code 2.1.207 with max effort level, `temperature = 1.0`, `top_p = 1.0`, `max_new_tokens = 128000`, and a 1M-token context window. We report the weighted average over 3 runs. Runs that fail to produce a score fall back to the official zero-shot base-model baseline score. For checks intended to prevent the use of third-party APIs, we removed the original pattern-matching-based checks, as they produced false positives when a local vLLM endpoint was accessed through the OpenAI SDK. Instead, we use an LLM agent to inspect solutions for external API usage.
- **SWE-Marathon:** We evaluate GLM-5.3 using Claude Code 2.1.207 with maximum effort level, `temperature = 1.0`, `top_p = 0.95`,

max_new_tokens = 128000, and a 1M-token context window. For strip-clone, the original anti-cheat checks used overly broad import detection that could reject valid implementations. We removed the affected checks and performed llm-based inspection instead to avoid false positives. For parameter-golf and trimul-cuda, changes to the NVIDIA wheels caused the Docker image builds to fail, so we added --extra-index-url https://pypi.org/simple to restore successful builds.

Citation

If you find GLM-5.3 useful in your research, please cite our technical report:

```
@misc{glm5team2026glm5vibecodingagentic,
  title={GLM-5: from Vibe Coding to Agent},
  author={GLM-5-Team and : and Aohan Zeng},
  year={2026},
  eprint={2602.15763},
  archivePrefix={arXiv},
  url={https://arxiv.org/abs/2602.15763}
}
```